

Video Transcript

Video 8: Redis: A Key-Value Database (07:08)

When it comes to database functionality, one of the most featureful, one of the ones that has the most things in it, even the name is very long, is relational database management systems. Now, at the other end of the spectrum, the simplest possible, the one that has the less friction for you to use would be key-value databases. You can almost think of them as the type of variable declaration that you do in your program, but in this case, you're getting the benefits of a server. When it comes to the database that we're going to be using, we're going to be using the open source project called 'Redis'. And this is an in-memory key-value pair data store, a database. It is very performant as you can imagine, it goes into memory, not all the way down to disk. So, it has those tremendous advantages of speed. And because of it, it is frequently used as an additional layer on top of other databases. So, let's say you're using MySQL. If you wanted to get more speed, more performance, you would add an additional layer using Redis. You could do the same thing with a document data store as well. And so this is one that is frequently used at other times that is used purely as a database that is with nothing else.

Now, in our case, we're going to do the same thing that we did with the RDBMS, that we did with the document store. In this case, we're going to create a container of the key-value pair data store, and in this case, we're going to do that with Redis. You can see here the command that we're going to enter, we have Docker run. The image is going to be Redis, the port is 6379, and the container name is going to be some Redis, and we're going to run that container detached.

So, as you can see here, I have my Docker dashboard, I've cleared out all my images and then I've gone ahead and pasted the same command instruction there that we had in our slides. Now, I'm going to go ahead and create it. You can see there that it is pulling from the Docker registry and it has created that instance. You can see here also within my dashboard that it is up and running. Now that we've created our container, I'm going to go ahead and drag and drop a folder from my desktop into my editor, and it looks like that did not go through. Let me try again.

And it looks like I already have that open. I'm going to go ahead and close it just to make sure I have the right one. You can see there that the color at the bottom of the editor turns purple. Now, I'm going to go ahead and drag and drop. Let's go ahead and open up here the file. So, we

can see that we have two files. The very first one is the write file. We'll read there in alphabetical order, but the first one we're going to work on is going to be write, and we will be writing to that Redis container.

And then we're going to use the second one to read what we write in. So, let's go ahead and open up that file. And as you can see here, the very first thing that we get is that we get the driver and we need to install that driver first. Let's go ahead and do that. Now, I'm going to grab there, pip install Redis, and then going to enter that at the command line. And as you can see in this environment, that has already been installed. If that wasn't the case, you would see the installation at that point.

So, let's go ahead and close the console and let's go on to the next line. And as you can see there, I am now using that driver to connect to the database. Note that I'm connecting locally, so that's localhost to my local container. The default port is 6379 and my database is the database zero. Again, very simple, very trim. The next thing that we're going to do is that we are going to create a timestamp, as you can see there, as we have done before, and now we're going to push that value into the database.

Note that we are not defining schema, we're not defining tables, collections, anything else. We're simply writing into the database, and in this case, under the handle of nums. Let's go ahead and save that file. Open up the console and let's go ahead and run that file. That's right. And let me make sure I have python3 here. And as you can see there, that went through. Now, let's go ahead and take a look at the read file. As you can see there we start once again with the driver. The next thing that we do is the connection to the database.

Then after that, we're simply reading the content that we have within it. So, let's go ahead and save the file. Let's go ahead and open up the console. In this case, I'm going to go ahead and run 3, I mean, Python 3 read. And as you can see there, we've written, we've read back the value that we added. So, let me go ahead and do a few more writes. And now let's go ahead and read those back. And as you can see there, we get the values. So, out of the databases that we have tried, you can see here that this is by far the simplest, the less syntax to write, the less options that are involved with creating and storing some information and retrieving it as well.

So, now it's your turn. Make sure that you can create the containers, make sure you can write and read from them. You might also want to try some of the visual managers for this, the graphical user interfaces for this database. This is the one that is available for Redis, you can

download it and install it and run it locally and take a look inside of your database. Now, just as a reminder, this is a very performant database and it is one that is used frequently on top of all other databases to improve scalability, to improve speed, and to have a better user experience.

It is also one of the friendliest when it comes to the developer experience and being able to store information. Now, not everything needs to be a simple key-value pair where the value of that key is a simple string. You can have a more complex structure, so you could map the type of rich information that you store within document databases or within a relational database into the structure. And depending on your needs, this is something that is done sometimes. So, once again, now it is your turn.

